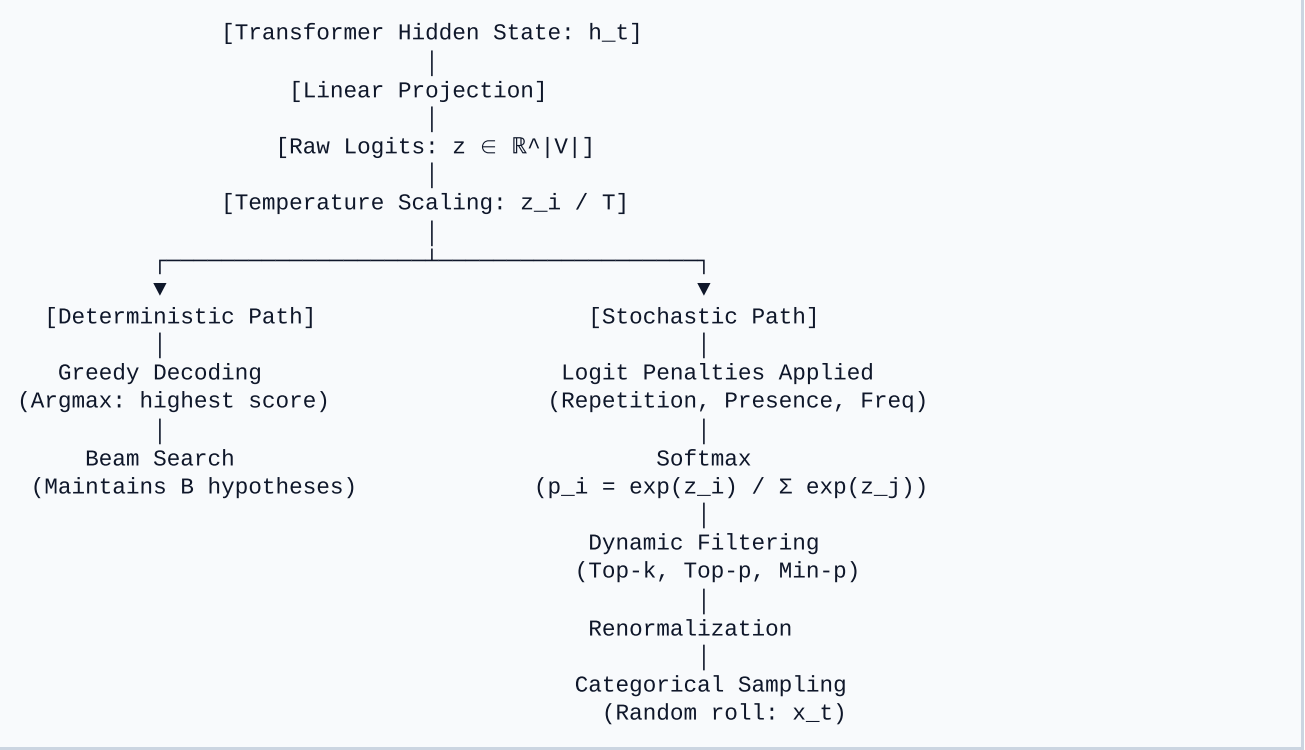
TECHNICAL REFERENCE & CURRICULUM GUIDE

Module 5: Decoding Strategies & Model Families

Module 5 covers the mechanics of token sampling, generation steering, comparative foundation model architectures, open-weight governance, test-time reasoning compute, and enterprise model selection.

5.1 Decoding Strategies: Math, Sampling Dynamics, and Output Impact

At each autoregressive generation step t, the Transformer's final linear projection produces a raw unnormalized vector of scores over the entire vocabulary V, called **logits** $\boldsymbol{z} = [z_1, z_2, \ldots, z_{|V|}] \in \mathbb{R}^{|V|}$. The decoding strategy determines how the model converts these continuous logits into a discrete token choice $x_t \in V$.



1. Greedy Decoding

Greedy decoding greedily selects the single token with the highest raw logit (maximum probability):

$$x_t = \arg\max_{i \in V} z_i = \arg\max_{i \in V} P(x_i \mid x_{<t})$$

- **Computational Complexity:** *O(1)* sampling overhead per step.
- **Characteristics:** 100% deterministic (zero randomness).
- **Failure Modes:** Prone to **repetition loops** (e.g., *"the car was fast, fast, fast..."*) and semantic traps. Because it never explores paths with lower immediate probabilities, it misses globally optimal sequences whose high-probability tokens appear later in the sentence.

2. Beam Search

Instead of committing to a single token, Beam Search maintains a fixed number B (the **beam width**) of the highest-scoring candidate sequences (hypotheses) at each step:

$$\text{Score}(x_1, \ldots, x_t) = \sum_{i=1}^{t} \log P(x_i \mid x_{<i})$$

- **Mechanics:** At step t, the model expands all B paths into $B \times |V|$ possible continuations, computes their cumulative log-probabilities, prunes the pool down to the top B candidates, and repeats until an end-of-sequence (`<eos>`) token is emitted.
- **Length Normalization:** Naive log-probability sums penalize longer sequences ($P \leq 1 \Rightarrow \log P \leq 0$). Beam search applies a length penalty:

$$\text{Score}_{\text{norm}}(Y) = \frac{\sum_{i=1}^{|Y|} \log P(x_i \mid x_{<i})}{|Y|^{\alpha}}, \quad \alpha \in [0.6, 0.8]$$

- **Why Modern LLMs Don't Use Beam Search for Open-Ended Generation:** Beam search optimizes for maximum global likelihood. In human language, the highest-probability path contains generic, repetitive, and bland phrasing ("The curious case of neural text degeneration", Holtzman et al.). Beam search remains widely used for **deterministic, constrained sequence-to-sequence tasks** (e.g., machine translation, Whisper speech-to-text, molecule synthesis), but is almost universally avoided for conversational reasoning and creative generation.

3. Temperature Scaling

Temperature $T \in [0, \infty)$ is a thermodynamic hyperparameter applied directly to the logit vector prior to the Softmax operation:

$$P(x_i \mid x_{<t}; T) = \frac{\exp(z_i / T)}{\sum_{j=1}^{|V|} \exp(z_j / T)}$$

- **Low Temperature ($T \to 0$, e.g., $T = 0.1 - 0.2$):** Divides logits by a tiny fraction, amplifying numerical differences. Logit differences of $+2$ become $+20$, causing the Softmax curve to become sharply peaked. As $T \to 0$, the distribution collapses into an argmax Dirac delta function, equivalent to greedy decoding. Minimizes hallucinations in code syntax, math, and structured JSON output.
- **Baseline Temperature ($T = 1.0$):** Retains the raw, unmodified training distribution.

- **High Temperature ($T > 1.0$, e.g., $T = 1.5 - 2.0$):** Compresses logit differences toward zero ($z_i / T \to 0$). The Softmax distribution flattens into a uniform distribution where every token in the vocabulary has a nearly equal probability of selection. Yields high lexical novelty and brainstorming variation, but quickly leads to syntactically degraded gibberish and structural breakdown.

4. Top-k Sampling (Fan et al., 2018)

Top-k truncates the vocabulary distribution to only the k tokens with the highest unnormalized probabilities, zeroing out the remaining $|V| - k$ tokens before sampling:

$$V^{(k)} = \text{top-}k \text{ subset of } V \text{ sorted by } P(x_i \mid x_{<t})$$

$$P'(x_i) = \begin{cases} \dfrac{P(x_i)}{\sum_{j \in V^{(k)}} P(x_j)} & \text{if } x_i \in V^{(k)} \\ 0 & \text{otherwise} \end{cases}$$

- **Mathematical Flaw (Contextual Rigidity):** The candidate pool size k is static regardless of the shape of the probability distribution:
  - *Peaked Distribution (High Confidence):* When predicting the completion to `"The President of the United States lives in the White ___"`, the token `"House"` may have a 99.2% probability. A setting of $k = 50$ forces the model to include 49 improbable, noisy tail tokens into the sampling pool.
  - *Flat Distribution (Low Confidence / High Branching):* When predicting an adjective following `"The view was ___"`, there may be 200 equally valid descriptors. A setting of $k = 10$ arbitrarily cuts off 190 natural, high-quality continuations.

5. Top-p (Nucleus) Sampling (Holtzman et al., 2019)

Nucleus sampling introduces a dynamic candidate pool size by defining a cumulative probability threshold $p \in (0, 1]$ (commonly $0.85 - 0.95$). It sorts all vocabulary tokens in descending probability order and selects the smallest subset $V^{(p)}$ whose summed mass satisfies:

$$\sum_{x_i \in V^{(p)}} P(x_i \mid x_{<t}) \geq p$$

Probabilities within $V^{(p)}$ are then renormalized to sum to 1.0:

$$P'(x_i) = \frac{P(x_i)}{\sum_{x_j \in V^{(p)}} P(x_j)}, \quad \forall\, x_i \in V^{(p)}$$

- **Dynamic Behavior:** When the model is confident, the nucleus shrinks to 1 or 2 tokens. When the context is ambiguous, the nucleus expands to include dozens of tokens, adapting candidate volume to model certainty.

5.2 Advanced Sampling Controls: Min-p, Repetition, and Frequency/Presence Penalties

While Temperature and Top-p manage overall distribution variance, production engines apply secondary logit filters to suppress degenerate repetition and dynamically trim the probability tail.

1. Min-p Sampling (The 2025–2026 Nucleus Replacement)

Although Top-p dynamically adapts its candidate pool, setting $p = 0.9$ can still retain very low-probability tokens when the leading token has a high probability (e.g., if token 1 has $P = 0.88$, Top-p will still include token 2 even if its probability is only 0.02).

Min-p establishes a dynamic probability floor scaled directly against the probability of the **top-ranked token** $P_{max} = \max_j P(x_j)$:

$$V^{(\text{min-}p)} = \{ x_i \in V \mid P(x_i) \geq \text{min_}p \times P_{\max} \}$$

- **Mathematical Mechanics:** If $\text{min_}p = 0.05$ (5% of top token):
  - *High Confidence Case:* If $P_{max} = 0.90$, any token with $P(x_i) < 0.05 \times 0.90 = 0.045$ is removed. The candidate pool is aggressively pruned to 1 or 2 tokens.
  - *Low Confidence Case:* If $P_{max} = 0.10$, any token with $P(x_i) \geq 0.05 \times 0.10 = 0.005$ remains eligible. The pool expands to allow diverse options.
- **Production Advantage:** Min-p eliminates the noisy tail of Top-p while enabling developers to use higher temperatures ($T = 1.0 - 1.2$) without inducing syntactic degradation or out-of-distribution hallucinations.

2. Repetition Penalty (Keskar et al. / CTRL Formulation)

Modifies the raw logit z_i of any token that has already appeared in the prompt or generated context:

$$z_i' = z_i / \theta \ (\text{if } z_i > 0), \quad z_i' = z_i \times \theta \ (\text{if } z_i \leq 0), \quad \text{where } \theta > 1.0$$

- Multiplying negative logits by θ makes them more negative; dividing positive logits by θ brings them closer to zero.
- **Sensitivity:** Setting $\theta > 1.2$ severely penalizes structural words (e.g., punctuation, pronouns, syntax brackets), causing models to write ungrammatical, contorted sentences to avoid repeating words like `"the"` or `","`. Typical safe production values range between 1.05 and 1.15.

3. Frequency Penalty vs. Presence Penalty (OpenAI API Formulation)

Instead of multiplicative scaling, OpenAI and modern inference engines introduce linear, additive logit subtractions:

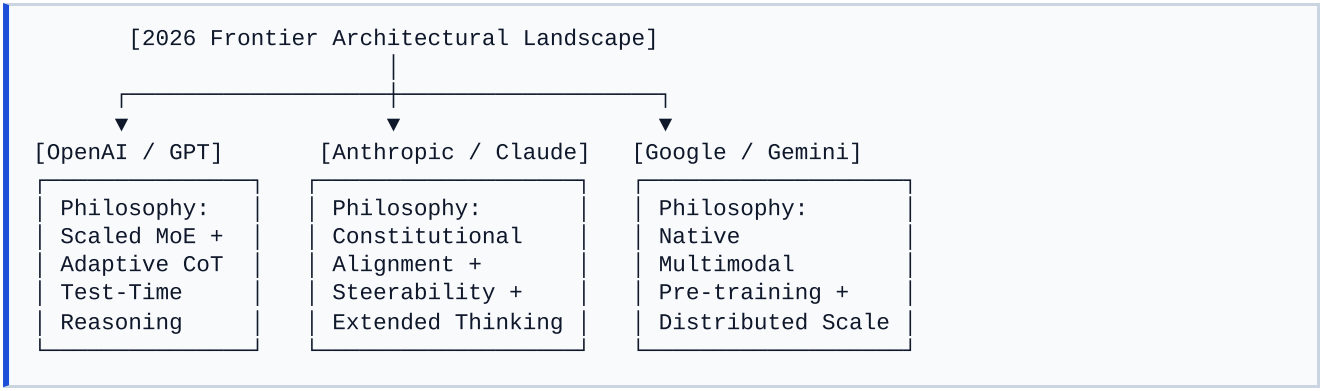
$$z_i' = z_i - (\text{count}(i) \cdot \mu_{\text{freq}}) - (\mathbb{I}(\text{count}(i) > 0) \cdot \mu_{\text{pres}})$$

- **Frequency Penalty ($\mu_{freq} \in [-2.0, 2.0]$):**
  - Scales **linearly with the exact number of times** token i has already been generated ($\text{count}(i)$).
  - *Impact:* Drives down the probability of overusing specific words in proportion to their frequency. Ideal for preventing repetitive phrases or expanding vocabulary diversity.
- **Presence Penalty ($\mu_{pres} \in [-2.0, 2.0]$):**
  - Applies a **one-time flat step penalty** if the token has appeared at least once ($\mathbb{I}(\text{count}(i) > 0)$), regardless of whether it appeared 1 time or 50 times.
  - *Impact:* Encourages the model to introduce fresh topics, themes, and entities rather than circling back to existing subjects.

Parameter	Application Stage	Mathematical Mechanism	Production Use Case	Typical Sweet Spot
Temperature (T)	Pre-Softmax	z_i / T	Global randomness & entropy control	0.0 – 0.2 (Code/Math), 0.7 – 0.9 (Prose)
Top-k	Logit Sorting	Discards indices where rank > k	Hard truncation of out-of-domain vocabulary	40 – 50 (often disabled if using Top-p/Min-p)
Top-p	Post-Softmax	Discards tokens past cumulative threshold $\sum p \geq p$	Dynamic tail trimming	0.80 – 0.95
Min-p	Post-Softmax	Discards $P(x_i) < \text{min_}p \times P_{\text{max}}$	Context-aware dynamic cutoff; replaces Top-p	0.05 – 0.10
Frequency Penalty	Pre-Softmax	$z_i - (\text{count} \times \mu_{\text{freq}})$	Penalizes recurrent overuse of specific tokens	0.1 – 0.5
Presence Penalty	Pre-Softmax	$z_i - (1 \times \mu_{\text{pres}})$	Promotes introduction of new topics/entities	0.1 – 0.5

5.3 Frontier Model Families Compared: GPT, Claude, and Gemini

Closed frontier models represent massive multi-trillion-parameter infrastructures optimized across different design philosophies, context handling methods, and tool-use ecosystems.



1. OpenAI (GPT-4o, GPT-5, o1, o3, o4-mini)

- **Architectural Philosophy:**
  - Massive Sparse Mixture-of-Experts (MoE) topologies combined with an **Adaptive Test-Time Reasoning Router**.
  - Unified omni-modal tokenization (interleaved audio, visual, and textual tokens trained end-to-end).
  - System convergence: Merging high-speed conversational models (4o) and deep reasoning models (o-series) under adaptive orchestration layers that automatically decide whether to respond immediately or route through extended chain-of-thought compute.
- **Context Windows & Economics:**
  - Standard API windows span 128K to 400K tokens.
  - Aggressive server-side KV prompt caching (up to 50% discount for cached prefixes), with low-latency speculative decoding engines.
- **Core Strengths:**
  - Tool-calling reliability, structured JSON output adherence, API ecosystem ubiquity, and test-time reasoning compute benchmarks in competitive mathematics and competitive programming (Codeforces).

2. Anthropic (Claude 3.5 Sonnet, Claude 3.7 Sonnet, Claude 4 / Opus)

- **Architectural Philosophy:**
  - Grounded in **Constitutional AI (RLAIF)** and mechanistic interpretability (Monosemanticity / Dictionary Learning of internal activations).
  - Steerability and safety: High adherence to complex, multi-page system prompts without instruction drift.
  - **Hybrid Extended Thinking & Computer Use:** Claude 3.7/4.x introduces controllable internal reasoning budgets (developers configure the exact maximum token allocation for internal chain-of-thought) alongside native OS-level GUI agent execution (processing screenshots and emitting mouse clicks, coordinates, and keystrokes).

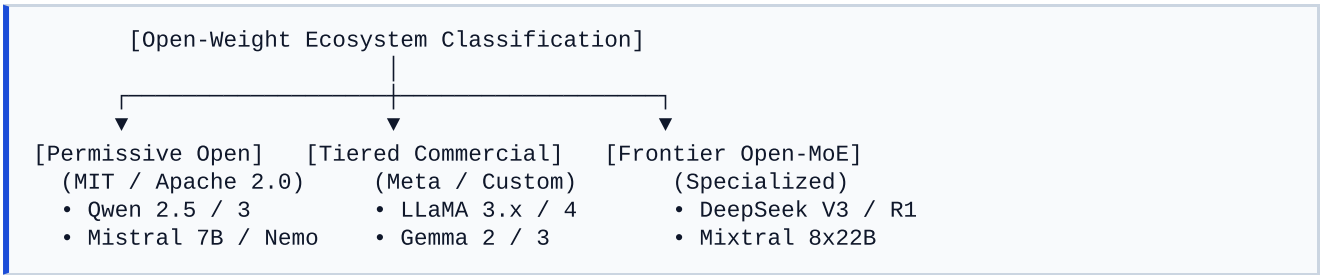
- **Context Windows & Economics:**
  - Production 200K token context window with consistent near-100% "Needle In A Haystack" (NIAH) retrieval accuracy across all token depths.
  - Explicit Prompt Caching (read/write pricing model offering up to 90% savings on 5-minute cache lifetimes).
- **Core Strengths:**
  - Software engineering (SWE-bench), refactoring large codebases, nuanced prose generation, character consistency, and adherence to strict developer constraints.

3. Google DeepMind (Gemini 1.5 Pro, Gemini 2.0 / 2.5, Gemini 3 Series)

- **Architectural Philosophy:**
  - **Native Multimodality from Inception:** Unlike pipelines that project image embeddings into a frozen text model via cross-attention bridges, Gemini interweaves video frames, multi-channel raw audio, images, and text during self-supervised pre-training.
  - Hardware Co-Design: Optimized for Google's proprietary Tensor Processing Unit (TPU v5p / TPU v6 Trillium) pods utilizing Megatron-style tensor and Ring Attention parallelisms.
- **Context Windows & Economics:**
  - **Production 1M to 2M+ Token Context Window.**
  - Capable of ingesting 1 hour of raw 1080p video, 11 hours of continuous audio, or code repositories over 60,000 lines in a single forward pass with zero retrieval chunking.
- **Core Strengths:**
  - Long-document synthesis, cross-modal video/audio comprehension, Google Workspace & Cloud ecosystem integration, and low-latency agentic loops (Gemini Flash).

5.4 Open-Weight Families: LLaMA, Mistral, Qwen, DeepSeek, and Gemma

Open-weight models provide downloadable weight checkpoints, allowing enterprises to achieve complete data sovereignty, eliminate third-party API dependencies, and customize architectures via fine-tuning.



1. Meta (LLaMA 3.1 / 3.3 / 4 Series: 8B, 70B, 405B)

- **Architecture:** Dense Transformers (8B, 70B, 405B) and MoE variants (LLaMA 4) utilizing GQA (8 KV heads), RoPE base frequencies scaled up to 500,000, and SwiGLU activations trained on 15T+ tokens.
- **Licensing: Meta LLaMA Community License Agreement**.
  - Free for research and commercial use *unless* the licensee's monthly active users (MAU) exceed 700 million at the time of model release (requiring a bespoke license from Meta).
  - Prohibits using LLaMA outputs to train or improve models other than LLaMA derivatives.
- **Deployment Implications:** The default base for fine-tuning, enterprise LoRA adapters, and on-premises vLLM serving. The 8B model fits on a single consumer GPU (RTX 4090 / 16GB VRAM quantized), while the 70B requires 2x 80GB A100/H100 GPUs in FP16 or 1x 80GB GPU in 4-bit (AWQ/EXL2).

2. Mistral AI (Mistral 7B, NeMo 12B, Mixtral 8x7B / 8x22B, Mistral Large 2)

- **Architecture:** Early pioneers of Sliding Window Attention (SWA), byte-fallback BPE, and production MoE (Mixtral routers dispatching each token to 2 out of 8 experts).
- **Licensing:** Bifurcated model:
  - Smaller models (Mistral 7B, NeMo, Mixtral 8x7B) released under **Apache 2.0** (fully permissive, commercial use with zero attribution or user-volume limits).
  - Flagship models (Mistral Large 2, Pixtral Large) released under **Mistral Commercial License** or research-only licenses, monetized through cloud endpoints and VPC deployments.
- **Deployment Implications:** Mixtral 8x7B requires 47B parameters of VRAM memory capacity (~90GB in FP16, ~48GB in 8-bit, ~24GB in 4-bit), but executes compute with the token latency of a 13B model.

3. Alibaba Cloud (Qwen 2.5 / Qwen 2.5-Coder / Qwen 3: 0.5B to 72B)

- **Architecture:** Dense models across granular sizes (0.5B, 1.5B, 3B, 7B, 14B, 32B, 72B) and specialized code/math models with context extensions up to 128K.
- **Licensing:** Primarily **Apache 2.0** for models up to 32B; 72B models released under the **Qwen License** (free commercial use up to 100M MAU).
- **Deployment Implications:** Qwen 2.5-Coder represents a top-tier open coding backbone, rivaling proprietary commercial engines on code completion, SQL generation, and function calling within self-hosted developer toolchains.

4. DeepSeek (DeepSeek-V3 / DeepSeek-R1)

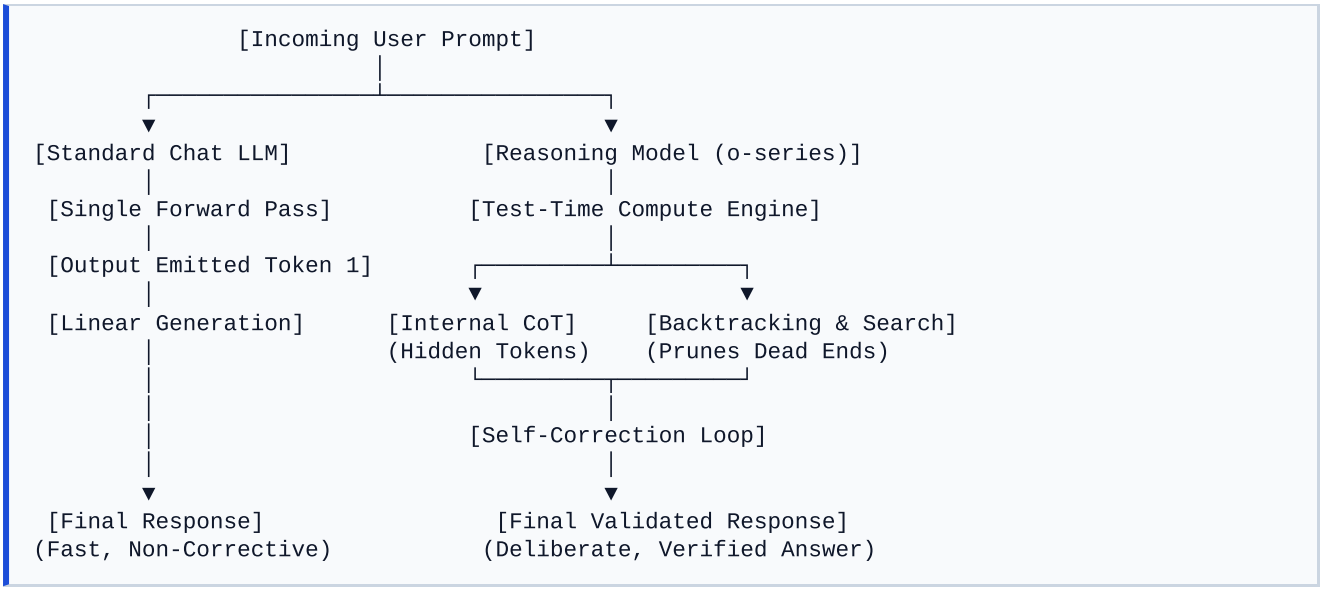
- **Architecture:** Highly optimized sparse MoE (671B total parameters, activating only 37B parameters per token).
  - **Multi-Head Latent Attention (MLA):** Compresses Keys and Values into a low-rank latent vector prior to caching, slashing KV-cache VRAM by over 80% compared to standard MHA/GQA.
  - **Auxiliary-Loss-Free Load Balancing:** Adjusts dynamic routing bias per expert without adding loss penalties that degrade perplexity.
  - **Multi-Token Prediction (MTP):** Predicts multiple sequential tokens per forward pass for speculative decoding acceleration.
- **Licensing: MIT License**. Exceptionally permissive: full commercial rights, royalty-free, with complete freedom to distill weights into other architectures.
- **Deployment Implications:** Due to its 671B parameter weight size, self-hosting requires multi-node FP8 clusters (e.g., minimum 8x 80GB H100/H800 GPUs). Widely adopted via decentralized third-party API providers (Together, Groq, Fireworks) at low cost per million tokens.

5. Google (Gemma 2 / Gemma 3: 2B, 9B, 27B)

- **Architecture:** Distilled from Gemini pre-training pipelines; uses alternating local sliding-window attention and global attention layers with logit soft-capping to prevent logit explosion during training.
- **Licensing: Gemma Terms of Use**. Permissive for research and commercial use, subject to Google's Acceptable Use Policy (restricting weapons, surveillance, etc.), with ownership of output models retained by the creator.
- **Deployment Implications:** Gemma 2B and 9B are heavily optimized for mobile and local edge devices (running via MediaPipe, ONNX, and Ollama on Apple Silicon and embedded NPUs).

5.5 Reasoning Models vs. Standard Chat Models

The release of models like OpenAI's **o1, o3, o4-mini**, Anthropic's **Claude Extended Thinking**, and Google's **Gemini Thinking** marks a paradigm shift: scaling compute at **inference time (test-time compute)** rather than relying solely on pre-training scale.



1. Operational & Mechanistic Differences

- **Standard Chat Models (Single-Trajectory Generation):**
  - When asked a complex puzzle, a standard LLM immediately begins outputting the first token of the final response.
  - *The Trap:* The model commits to a reasoning branch from token 1. If that branch leads to an arithmetic or logical dead end, the model cannot delete historical tokens due to causal attention masking. It is forced to confabulate and defend its earlier error.
- **Reasoning Models (Internalized Test-Time Search):**
  - Before emitting a single user-visible token, the model executes an **internal Chain-of-Thought (CoT)** loop, generating hundreds or thousands of hidden "thinking tokens".
  - *Search & Backtracking:* Trained via **Large-Scale Reinforcement Learning (RL)** with verifiable reward models (in code execution, mathematical proofs, and formal logic). When the model detects an internal contradiction during its thinking phase, it outputs tokens that explicitly say: *"Wait, this equation contradicts step 2, let me recalculate from scratch..."*, pruning the failed trajectory and exploring an alternative path.

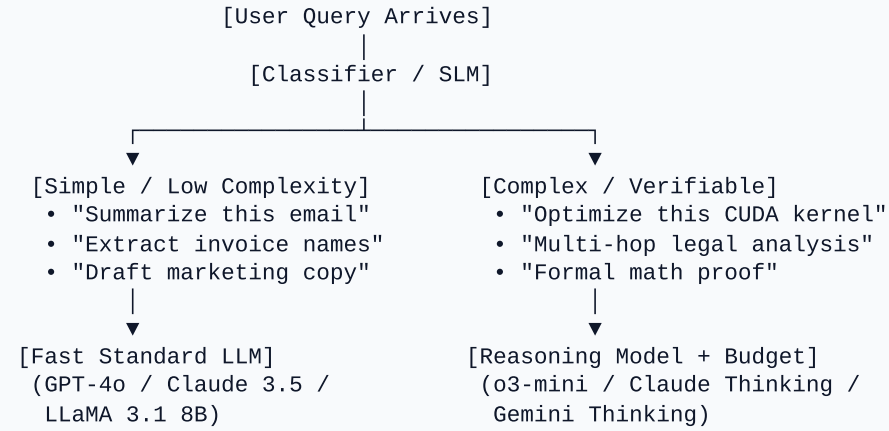
2. Operational Comparison Matrix

Operational Dimension	Standard Chat LLMs (e.g., GPT-4o, Claude 3.5)	Reasoning Models (e.g., o1, o3, Claude 3.7 Thinking)
Latency Profile	Low TTFT (Time-To-First-Token: 200ms – 1.5s)	High TTFT (5s to 60s+ spent generating internal thoughts)
Token Consumption	Billed strictly on visible Input + Output tokens	Billed on Input + **Hidden Thinking Tokens** + Output tokens
Prompt Engineering	Requires explicit CoT prompts (*"Think step-by-step"*)	CoT prompts degrade performance; plain, direct prompts work best
System Visibility	100% of generated tokens are visible	Thinking traces are encrypted, abstracted, or summarized

Operational Dimension	Standard Chat LLMs (e.g., GPT-4o, Claude 3.5)	Reasoning Models (e.g., o1, o3, Claude 3.7 Thinking)
Task Performance	Strong at creative prose, summarization, fast chat	Dominates competitive math, complex code debugging, legal reasoning

3. Production Architecture: Router Pattern

Reasoning models should **not** be deployed as blanket replacements for standard chat models. Doing so introduces unacceptable latency and inflates token billing by *5*× to *20*× on simple tasks.



5.6 Enterprise Decision Matrix: Capability, Cost, Latency, and Privacy

Choosing the correct foundation model requires balancing four competing engineering constraints: **Reasoning Capability**, **Cost per Transaction**, **P99 Latency (TTFT & Throughput)**, and **Data Privacy / Governance**.



The 2026 Production Tier Evaluation Matrix

Model Tier	Representative Models	Blended Cost (per 1M Tokens)	Latency (P50 TTFT / Speed)	Data Privacy & Residency	Optimal Production Workload
Tier 1: Frontier Reasoning	OpenAI o3 / o1, Claude 4 Extended Thinking	High ($15.00 – $60.00+)	Very Slow (TTFT 5s – 40s; 30–60 tps)	Closed Cloud: Subject to enterprise zero-retention (ZDP) contracts	Autonomous code architecture, multi-page contract reconciliation, edge-case debugging

Model Tier	Representative Models	Blended Cost (per 1M Tokens)	Latency (P50 TTFT / Speed)	Data Privacy & Residency	Optimal Production Workload
Tier 2: Flagship Omni / Multimodal	GPT-4o, Claude 3.5 Sonnet, Gemini 1.5 Pro	Moderate ($2.50 – $15.00)	Fast (TTFT 300ms – 800ms; 80–120 tps)	Closed Cloud: Encrypted in transit/rest; SOC2, HIPAA, GDPR compliant	Interactive enterprise agents, live video/audio streams, full-repo analysis
Tier 3: High-Efficiency Cloud SLMs	GPT-4o-mini, Claude 3.5 Haiku, Gemini 2.0 Flash	Low ($0.07 – $0.80)	Ultra-Fast (TTFT 150ms – 300ms; 150–250 tps)	Closed Cloud: ZDP enterprise options	High-volume RAG generation, text classification, data extraction, initial query routing
Tier 4: Self-Hosted Open-Weight Flagships	DeepSeek-V3 / R1, LLaMA 3.1 70B/405B	Compute-Bound (GPU leasing: ~$2 – $8 / hr per node)	Variable (Engine-dependent: vLLM / TensorRT-LLM 40–100 tps)	**Maximum:** 100% On-Premises, Air-Gapped, VPC private subnet (zero third-party egress)	Highly regulated environments (Defense, Healthcare, Banking, proprietary IP)
Tier 5: Edge / On-Device SLMs	LLaMA 3.2 1B/3B, Gemma 2 2B/9B, Qwen 2.5 7B	Zero variable token cost (Client hardware compute)	Instant (Runs locally on Apple Silicon, Snapdragon, edge NPU)	**Absolute:** Data never leaves local device memory	Local mobile autocomplete, personal assistant copilots, offline IoT devices

Systematic Framework for Model Selection

1. **Apply the Privacy Gate First:**
   - *Does compliance (HIPAA, ITAR, internal IP restrictions) prohibit customer data from exiting an isolated VPC or air-gapped data center?*
   - If **Yes** → Immediately discard commercial closed APIs. Deploy an open-weight model (LLaMA 3.1 70B, DeepSeek-V3, or Qwen 2.5) on private cloud GPU instances (AWS Bedrock private clusters, Azure Private Link, or local HGX H100s) via vLLM.
2. **Apply the Latency Gate Second:**
   - *Is the application a real-time, user-facing synchronous interface (e.g., search autocomplete, conversational voice assistant, inline IDE code suggestions)?*
   - If **Yes** → Discard reasoning models. The requirement demands TTFT $< 400\text{ms}$ and throughput $> 80\text{ tps}$. Select high-throughput models like Gemini 2.0 Flash, Claude 3.5 Haiku, or a local 8B GQA checkpoint.
3. **Apply the Complexity & Verification Gate Third:**
   - *Is the task a complex, multi-hop problem with an objectively verifiable answer (code refactoring, symbolic math, logical constraint satisfaction) where errors break production?*
   - If **Yes** → Route to a **Reasoning Model** (o3-mini, Claude Thinking). The cost and latency overhead are outweighed by the reduction in downstream human verification and bug remediation.
4. **Optimize for Cost via Cascading & Distillation:**
   - In steady-state production, run a **Cascade Architecture**: Send 100% of user queries to a fast, low-cost classifier (e.g., Tier 3 or a fine-tuned 3B model). Resolve 80% of standard retrieval and summarization queries at low token cost, routing only the 20% most ambiguous, dense, or high-risk prompts to flagship models or reasoning engines.